**Выполнил: Василенко И.А Группа: ПИа-090303-о24**

# ЛАБОРАТОРНАЯ РАБОТА №11

# Дисциплина: Основы программирования на Python Тема: Реляционные и нереляционные базы данных. Хранение данных и использование CDN

# Вариант 2: Библиотека

## Цель работы

Получить практические навыки проектирования и использования систем хранения данных в Python-приложениях: изучить принципы работы реляционных баз данных, освоить SQL, первичные и внешние ключи, связи между таблицами и нормализацию; разобрать операции SELECT, INSERT, UPDATE, DELETE, JOIN, группировку и агрегатные функции, транзакции; подключить Python-приложение к реляционной БД и организовать слой Repository; сравнить реляционный и нереляционный подходы; реализовать дополнительное хранилище на базе Redis и организовать кэширование с TTL и инвалидацией; изучить назначение CDN и организовать раздачу статических ресурсов через CDN.

## Постановка индивидуального задания (Вариант 2. Библиотека)

Предметная область — библиотека, основная сущность — Book (книга). Требуется модернизировать REST-приложение из лабораторной работы №10: данные о книгах должны храниться не в list, а в реляционной базе данных; обращения к БД выполняются через слой Repository; NoSQL-часть реализуется на Redis (история поиска и кэш); CDN-часть — обложки книг и статические файлы оформления.

В соответствии с заданием необходимо:

– спроектировать схему БД: не менее 3 таблиц, первичный ключ в каждой таблице, не менее 2 внешних ключей, связь 1:N и связь N:M через промежуточную таблицу;

– создать SQL-скрипт schema.sql и заполнить БД тестовыми данными (не менее 10 записей основной сущности, 5 записей связанных сущностей, записи для связи N:M);
– написать не менее 15 SQL-запросов, включая SELECT, WHERE, ORDER BY, JOIN, LEFT JOIN, GROUP BY, HAVING, COUNT, AVG/SUM, INSERT, UPDATE, DELETE, подзапрос, запрос с несколькими условиями и запрос со связью нескольких таблиц;
– определить не менее двух индексов и объяснить их назначение;
– перевести все endpoints лабораторной работы №10 на работу с БД через Repository, используя только параметризованные запросы;
– реализовать бизнес-операцию, требующую транзакции (выдача и возврат книги);
– реализовать Redis-кэш по паттерну cache-aside с TTL и инвалидацией, а также историю поиска;
– разместить не менее трёх статических ресурсов и получить их через CDN, исследовав заголовки и поведение кэша;
– написать не менее 10 автоматических тестов на pytest.

**Краткие теоретические сведения**

Реляционная база данных представляет данные в виде таблиц, связанных между собой ключами. Первичный ключ (PRIMARY KEY) однозначно идентифицирует строку таблицы, внешний ключ (FOREIGN KEY) ссылается на первичный ключ другой таблицы и обеспечивает ссылочную целостность. Основные типы связей: один к одному (1:1), один ко многим (1:N) и многие ко многим (N:M). Связь N:M в реляционной модели реализуется через промежуточную таблицу, содержащую внешние ключи обеих сторон.

Нормализация уменьшает дублирование данных и устраняет аномалии вставки, обновления и удаления. Первая нормальная форма требует атомарности значений в ячейках, вторая — зависимости неключевых атрибутов от полного первичного ключа, третья — отсутствия транзитивных

зависимостей между неключевыми атрибутами. В спроектированной схеме сведения об авторе вынесены в отдельную таблицу authors, а теги — в tags и book_tags, что соответствует 3НФ.

SQL — декларативный язык работы с реляционными данными: SELECT выбирает строки, INSERT добавляет, UPDATE изменяет, DELETE удаляет. Оператор JOIN соединяет таблицы по условию, GROUP BY группирует строки, агрегатные функции COUNT, SUM, AVG, MIN, MAX вычисляют статистику, HAVING фильтрует уже сгруппированные данные, а подзапрос позволяет использовать результат одного запроса внутри другого.

Транзакция — набор операций, выполняемый как единое целое. Свойства ACID: атомарность (все операции выполняются или ни одна), согласованность (БД переходит из одного корректного состояния в другое), изолированность (параллельные транзакции не мешают друг другу) и долговечность (после COMMIT изменения сохраняются). При ошибке выполняется ROLLBACK, и данные возвращаются к состоянию до начала транзакции.

Индекс — дополнительная структура ускорения поиска, которая хранит упорядоченные значения столбца со ссылками на строки. Индекс ускоряет SELECT, но замедляет INSERT, UPDATE и DELETE и занимает место на диске, поэтому его создают под конкретные запросы, а не на каждый столбец.

SQL-инъекция — атака, при которой пользовательские данные попадают в текст SQL-запроса и меняют его смысл. Приём вида «SELECT * FROM users WHERE name = '" + name + "'» опасен: значение "' OR '1'='1" превращает условие в всегда истинное. Параметризованные запросы (placeholders '?') передают значения отдельно от текста запроса, поэтому данные не могут стать частью SQL-кода.

NoSQL объединяет хранилища, не использующие классическую реляционную модель: документные (MongoDB), ключ-значение (Redis), колоночные (Cassandra), графовые (Neo4j). Redis хранит данные в памяти в виде ключ-значение, поддерживает TTL и атомарные операции INCR, поэтому часто применяется как кэш, хранилище сессий и счётчиков.

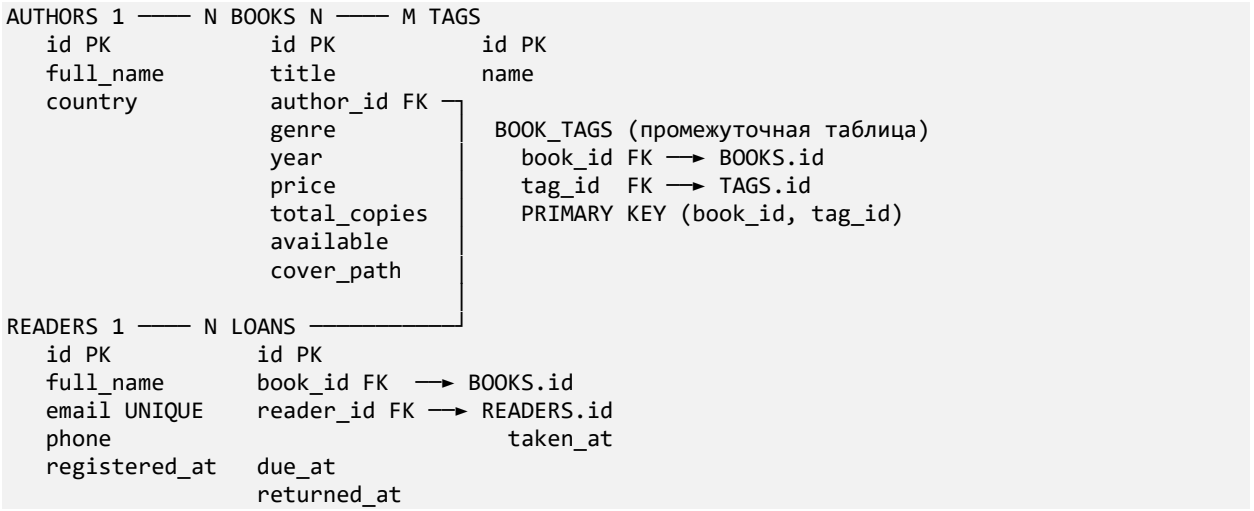
Кэширование по паттерну cache-aside: приложение сначала проверяет кэш, при промахе читает данные из основной БД, записывает их в кэш с TTL и возвращает клиенту. TTL ограничивает время жизни записи, а cache invalidation удаляет или обновляет запись при изменении данных, чтобы клиент не получил устаревшее значение.

CDN (Content Delivery Network) — распределённая сеть edge-серверов, которые отдают статический контент ближе к пользователю и снижают нагрузку на origin-сервер. Кэшированием на edge управляет заголовок Cache-Control (например, «public, max-age=86400»), а признаками попадания в кэш служат заголовки X-Cache и Age. Через CDN передают изображения, CSS, JavaScript, видео и шрифты; изменяемые данные приложения (цены, остатки) должны поступать из БД.

## Проектирование базы данных

Для предметной области «Библиотека» спроектирована схема из шести таблиц: authors, books, readers, loans, tags и book_tags. Схема содержит пять внешних ключей, связь 1:N (authors → books, readers → loans, books → loans) и связь N:M (books ↔ tags через промежуточную таблицу book_tags).

*Схема 11.1 — ER-диаграмма базы данных «Библиотека»*




Назначение таблиц и ограничения целостности приведены в таблице 11.1.

*Таблица 11.1 — Назначение таблиц и ограничения целостности*

| Таблица | Назначение | Ключи и ограничения |
|---|---|---|

| | | |
|---|---|---|
| authors | Справочник авторов | id PK; full_name UNIQUE, NOT NULL |
| books | Библиотечный фонд | id PK; author_id FK → authors(id); CHECK (year 1…2100), CHECK (price > 0), CHECK (available_copies ≤ total_copies) |
| readers | Читатели | id PK; email UNIQUE, NOT NULL |
| loans | Выдачи книг | id PK; book_id FK → books(id) ON DELETE CASCADE; reader_id FK → readers(id); CHECK (due_at ≥ taken_at) |
| tags | Теги произведений | id PK; name UNIQUE |
| book_tags | Связь N:M books ↔ tags | PRIMARY KEY (book_id, tag_id); два внешних ключа |

Схема обоснована с точки зрения нормализации: ФИО автора хранится один раз в таблице authors (устранено дублирование в books), теги вынесены в отдельный справочник, а связь книги и тега описана в промежуточной таблице. Все неключевые атрибуты зависят только от первичного ключа своей таблицы, транзитивных зависимостей нет — схема находится в 3НФ.

База данных создаётся скриптом schema.sql; тестовые данные (12 книг, 6 авторов, 5 читателей, 8 выдач, 8 тегов, 24 связи N:M) загружаются из seed.sql. Структура созданной БД показана на рисунке 11.1.

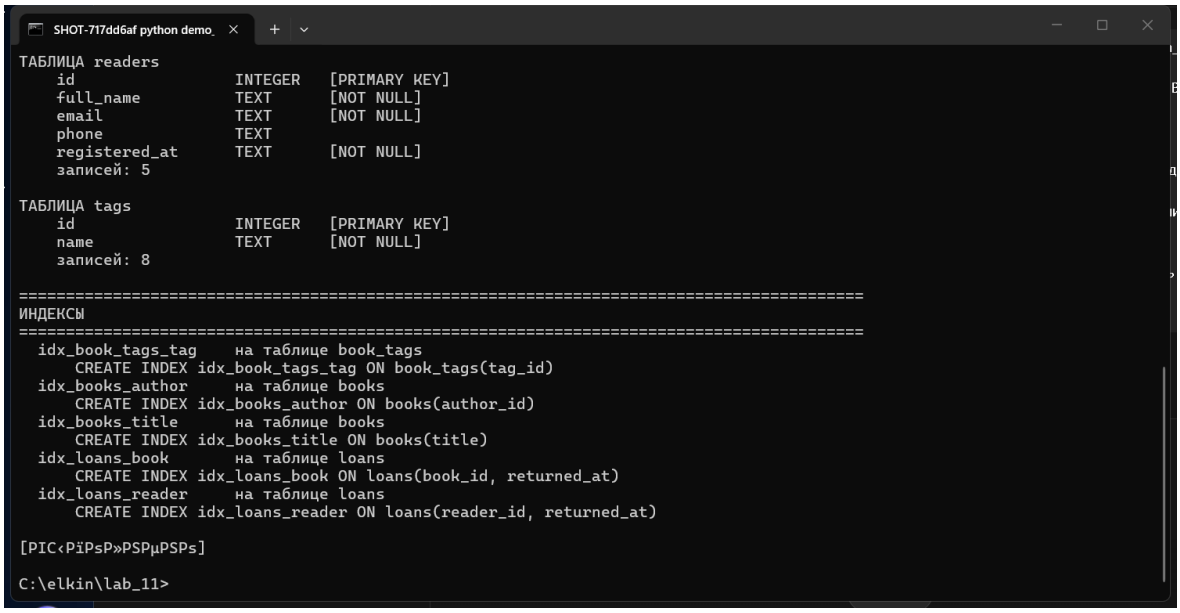



Рисунок 11.1 — Структура базы данных: таблицы, ключи, количество записей и индексы

## SQL-запросы

В файле queries.sql подготовлено 15 запросов, охватывающих все обязательные конструкции задания. Перечень запросов приведён в таблице 11.2.

*Таблица 11.2 — Перечень SQL-запросов и использованных конструкций*

| № | Конструкция | Назначение запроса |
|---|---|---|
| 1 | SELECT | выборка всех книг фонда |
| 2 | WHERE | книги дешевле 600 рублей, кроме антиутопий |
| 3 | ORDER BY | сортировка фонда по цене по убыванию |
| 4 | JOIN | книги вместе с авторами (связь 1:N) |
| 5 | LEFT JOIN + COUNT | читатели и число взятых книг, включая читателей без выдач |
| 6 | GROUP BY | количество книг по жанрам |
| 7 | HAVING | авторы, у которых больше двух книг в фонде |
| 8 | COUNT | количество невозвращённых выдач |
| 9 | AVG, SUM | средняя цена книги и стоимость фонда |
| 10 | подзапрос | книги дороже средней цены фонда |
| 11 | несколько условий | активные выдачи с просрочкой на заданную дату |
| 12 | JOIN трёх таблиц | книги, авторы и теги через book_tags |
| 13 | INSERT | регистрация нового читателя |
| 14 | UPDATE | повышение цены книг жанра «антиутопия» на 10 % |
| 15 | DELETE + подзапрос | удаление читателя, не бравшего книг |

*Листинг 11.1 — Примеры запросов из queries.sql*

```
-- 4. JOIN: книги вместе с авторами (связь 1:N)
SELECT b.title, a.full_name, a.country
FROM books AS b
JOIN authors AS a ON a.id = b.author_id
ORDER BY a.full_name, b.title;


-- 7. HAVING: авторы, у которых больше двух книг в фонде
SELECT a.full_name, COUNT(b.id) AS books_count
FROM authors AS a
JOIN books AS b ON b.author_id = a.id
GROUP BY a.id, a.full_name
HAVING COUNT(b.id) > 2
ORDER BY books_count DESC;


-- 12. Связь нескольких таблиц через промежуточную таблицу
SELECT b.title, a.full_name, t.name AS tag
FROM books AS b
JOIN authors AS a    ON a.id = b.author_id
JOIN book_tags AS bt ON bt.book_id = b.id
JOIN tags AS t       ON t.id = bt.tag_id
WHERE t.name = 'фантастика'
ORDER BY b.title;
```

Результаты выполнения запросов приведены на рисунках 11.2 и 11.3.

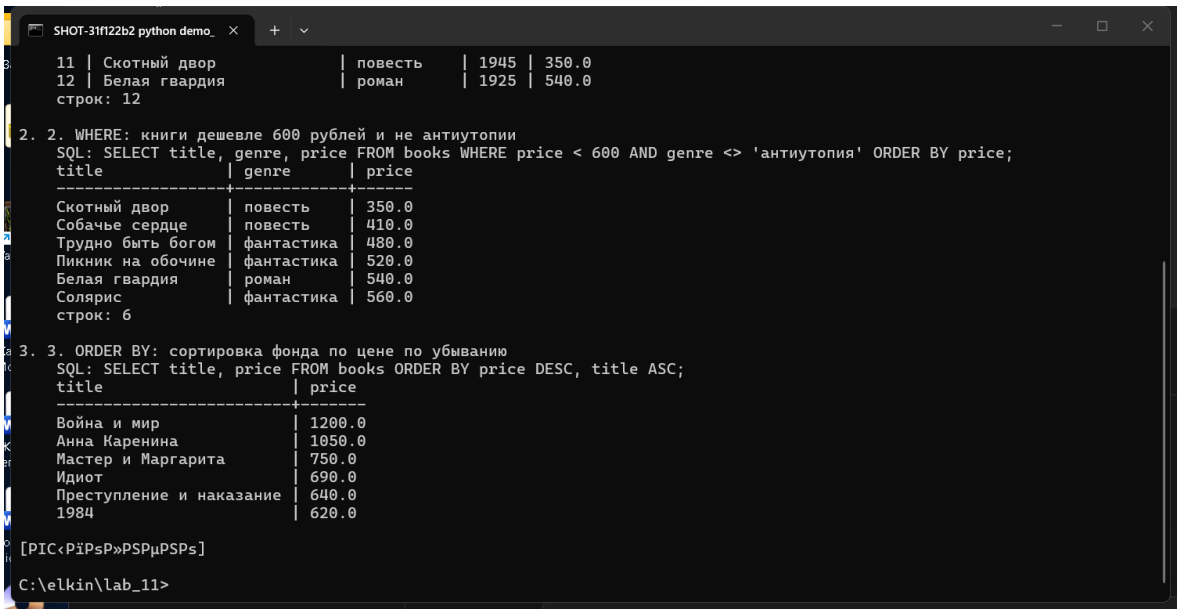



Рисунок 11.2 — Выполнение SQL-запросов № 1–6

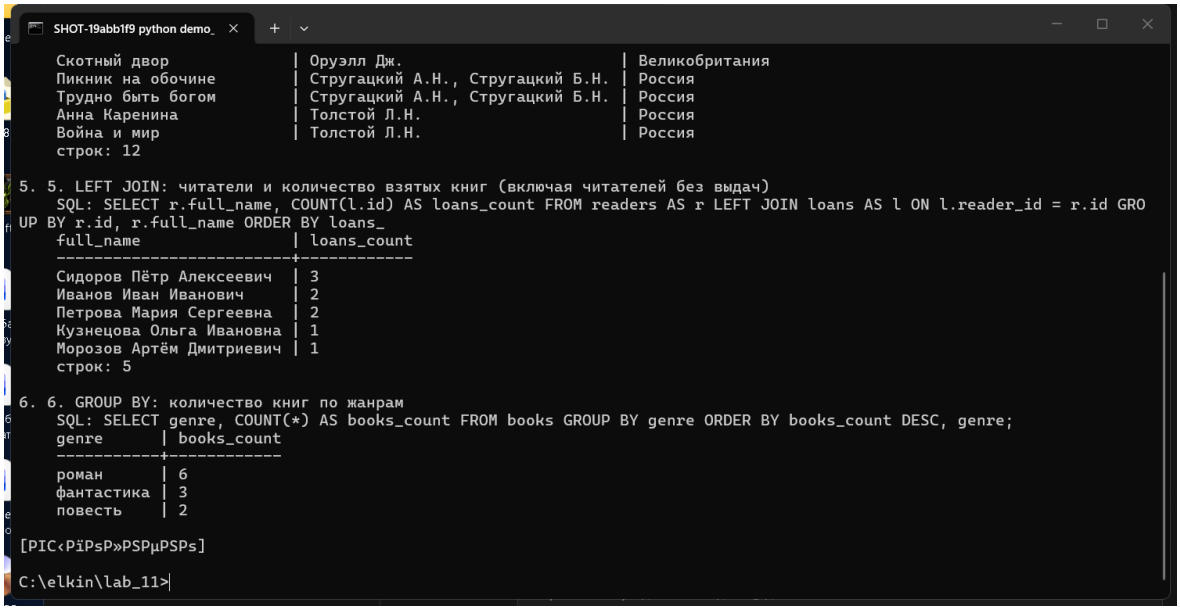



Рисунок 11.3 — Выполнение SQL-запросов № 7–15

## Индексы

В схеме создано пять индексов. Каждый индекс предназначен для конкретного запроса приложения, а не «на всякий случай» (таблица 11.3).

*Таблица 11.3 — Индексы базы данных и их назначение*

| Индекс | Столбцы | Для какого запроса |
|---|---|---|
| idx_books_title | books(title) | поиск и сортировка по названию: WHERE title = ? / ORDER BY title |
| idx_books_author | books(author_id) | фильтрация по автору и JOIN authors |
| idx_loans_reader | loans(reader_id, returned_at) | GET /readers/{id}/loans, поиск активных выдач |
| idx_loans_book | loans(book_id, returned_at) | проверка активных выдач конкретной книги |

| | | |
|---|---|---|
| idx_book_tags_tag | book_tags(tag_id) | обратная выборка книг по тегу (связь N:M) |

Назначение индексов проверено планами выполнения (EXPLAIN QUERY PLAN):

*Листинг 11.2 — Использование индексов в плане запроса*

```
EXPLAIN QUERY PLAN SELECT * FROM loans WHERE reader_id = 2 AND returned_at IS NULL;
   SEARCH loans USING INDEX idx_loans_reader (reader_id=? AND returned_at=?)


EXPLAIN QUERY PLAN SELECT * FROM books WHERE title = 'Война и мир';
   SEARCH books USING INDEX idx_books_title (title=?)
```

Индекс не создаётся на каждом столбце: каждая дополнительная структура увеличивает размер БД и замедляет INSERT, UPDATE и DELETE, так как её нужно поддерживать в актуальном состоянии. Кроме того, индекс не используется, если условие начинается с шаблона LIKE '%текст%' или если столбец имеет низкую селективность (например, поле с двумя значениями). Поэтому индексы созданы только под реальные запросы приложения: поиск по названию, фильтрация по автору и выборка выдач по читателю и книге.

## Транзакции

Бизнес-операция «выдача книги» изменяет две таблицы: в loans добавляется запись о выдаче, а в books уменьшается количество свободных экземпляров. Если выполнить только первую операцию, читатель получит книгу, которой нет в наличии, поэтому обе операции объединены в транзакцию: при ошибке на любом шаге выполняется ROLLBACK.

*Листинг 11.3 — Менеджер транзакций (db.py)*

```
@contextmanager
def transaction(self):
    connection = self.connect()
    try:
        connection.execute("BEGIN IMMEDIATE")
        yield connection
        connection.commit()      # COMMIT — блок завершился без исключения
    except Exception:
        connection.rollback()    # ROLLBACK — любое исключение внутри блока
        raise
    finally:
        connection.close()
```

*Листинг 11.4 — Транзакция выдачи книги (service.py)*

```
with self.database.transaction() as connection:
    row = connection.execute(
```

```
        "SELECT available_copies FROM books WHERE id = ?", (book_id,)
    ).fetchone()
    if row is None:
        raise NotFoundError(f"Book with id={book_id} not found")
    if row["available_copies"] <= 0:
        raise ConflictError("Book has no available copies")

    loan_id = self.loans.create(connection, book_id, reader_id, taken_at, due_at)
    cursor = connection.execute(
        "UPDATE books SET available_copies = available_copies - 1"
        " WHERE id = ? AND available_copies > 0", (book_id,)
    )
    if cursor.rowcount != 1:
        raise ConflictError("Book copy was taken by another reader")
```

На рисунке 11.4 показаны три эксперимента: успешная выдача (COMMIT), попытка выдать книгу, все экземпляры которой уже на руках (ROLLBACK, состояние БД не изменилось), и возврат книги (COMMIT, оба UPDATE применились вместе).

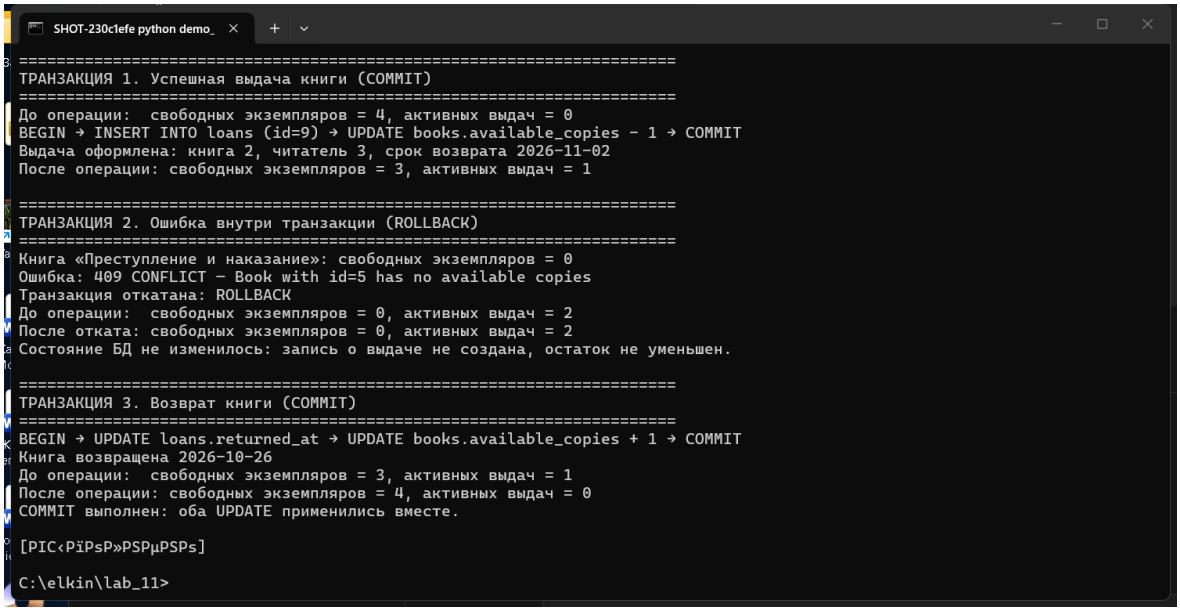



Рисунок 11.4 — Транзакции: COMMIT при выдаче, ROLLBACK при отсутствии экземпляров, COMMIT при возврате

## Интеграция с Python

Приложение построено по той же схеме, что и в лабораторной работе №10, но хранилище в памяти заменено базой данных:

*Листинг 11.5 — Архитектура приложения*

```
HTTP Client
     │ HTTP
     ▼
Routes (routes.py)      разбор запроса, коды ответов
     ▼
Service (service.py)    бизнес-логика, валидация, кэш
     ▼
```

```
Repository (repository.py)  SQL-запросы, параметры, транзакции
     ▼
SQLite (library.db)     таблицы, индексы, ограничения
```

Все SQL-запросы выполняются только в слое Repository и только с параметрами. Пример параметризованного INSERT:

*Листинг 11.6 — Параметризованный запрос (repository.py)*

```
query = '''
    INSERT INTO books (title, author_id, genre, year, price,
                       total_copies, available_copies, cover_path)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
'''
params = (data["title"], data["author_id"], data["genre"], data["year"],
          data["price"], data["total_copies"], available, data.get("cover_path"))
with self.database.connection() as connection:
    cursor = connection.execute(query, params)
```

Список endpoints, переведённых на работу с БД, приведён в таблице 11.4.

*Таблица 11.4 — Endpoints REST API и выполняемые SQL-операции*

| Метод | Endpoint | Операция SQL |
|---|---|---|
| GET | /books | SELECT с фильтрами и сортировкой |
| GET | /books/{id} | SELECT по первичному ключу (через кэш) |
| GET | /books/{id}/details | SELECT с JOIN автора и GROUP_CONCAT тегов |
| GET | /books/search?q= | SELECT по названию + INCR счётчика в Redis |
| GET | /books/search-history | чтение истории поиска из Redis |
| POST | /books | INSERT |
| PUT | /books/{id} | UPDATE |
| PATCH | /books/{id} | UPDATE + инвалидация кэша |
| DELETE | /books/{id} | DELETE |
| PUT | /books/{id}/tags | INSERT в book_tags (связь N:M) |
| GET | /loans | SELECT с JOIN books и readers |
| POST | /loans | транзакция: INSERT + UPDATE |
| POST | /loans/{id}/return | транзакция: UPDATE + UPDATE |
| GET | /stats | GROUP BY, HAVING, COUNT, AVG |
| GET | / | страница каталога (HTML-шаблон) |
| GET | /catalog | SELECT книг с ФИО автора и ссылкой на обложку CDN |

Для демонстрации работы API используется консольный клиент client.py с меню (рисунок 11.5).

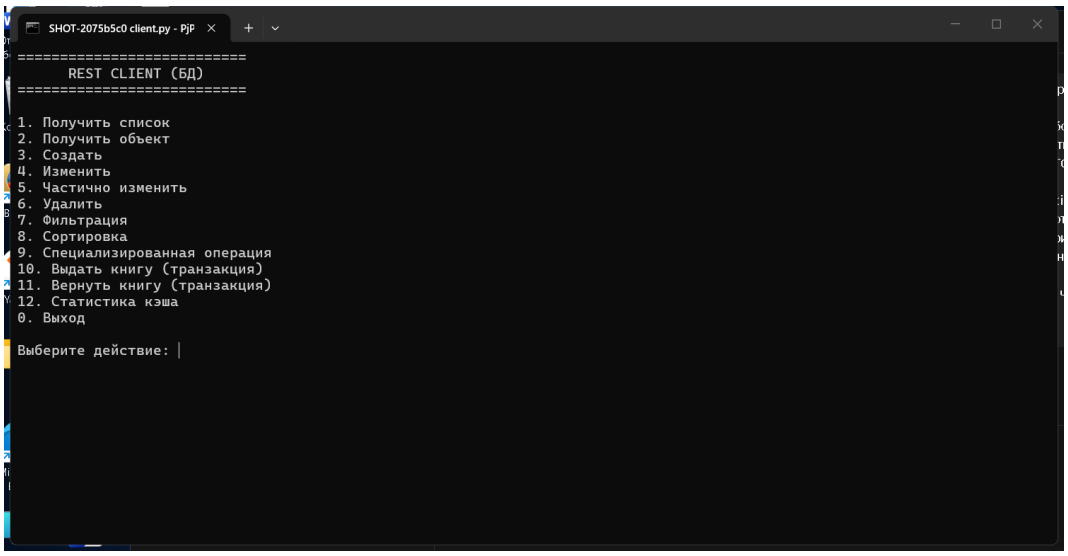



Рисунок 11.5 — Меню консольного клиента REST API

На рисунке 11.6 приведены список книг (GET /books) и чтение книги по идентификатору (GET /books/1, первый запрос — промах кэша, второй — попадание).

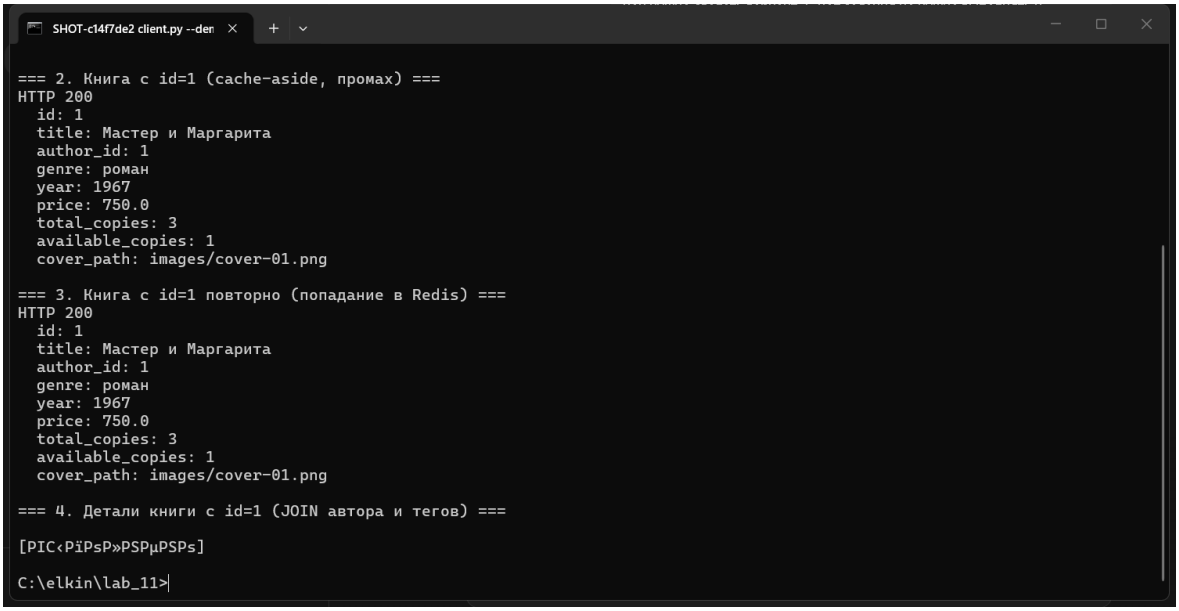



Рисунок 11.6 — Получение списка книг и чтение объекта по идентификатору

На рисунке 11.7 показаны создание (POST), полное изменение (PUT), частичное изменение (PATCH) и удаление (DELETE) книги. При изменении и удалении ключ кэша инвалидируется.

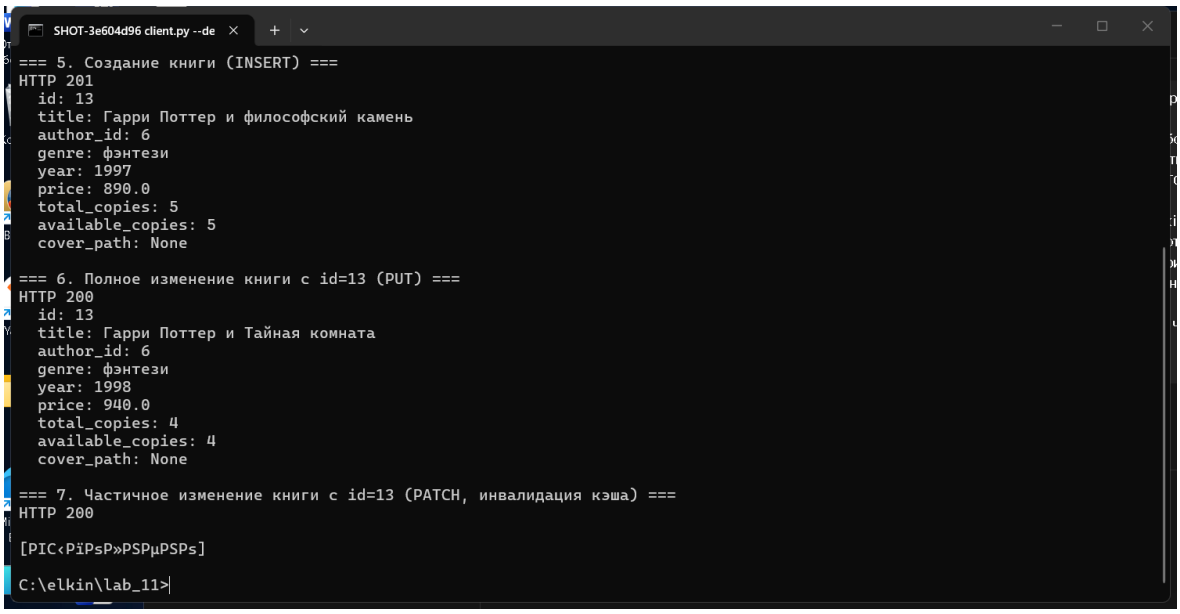



Рисунок 11.7 — Создание, изменение и удаление записи через REST API

На рисунке 11.8 показаны операции выдачи и возврата книги, выполняемые в транзакции: после выдачи количество доступных экземпляров уменьшается, после возврата — восстанавливается.

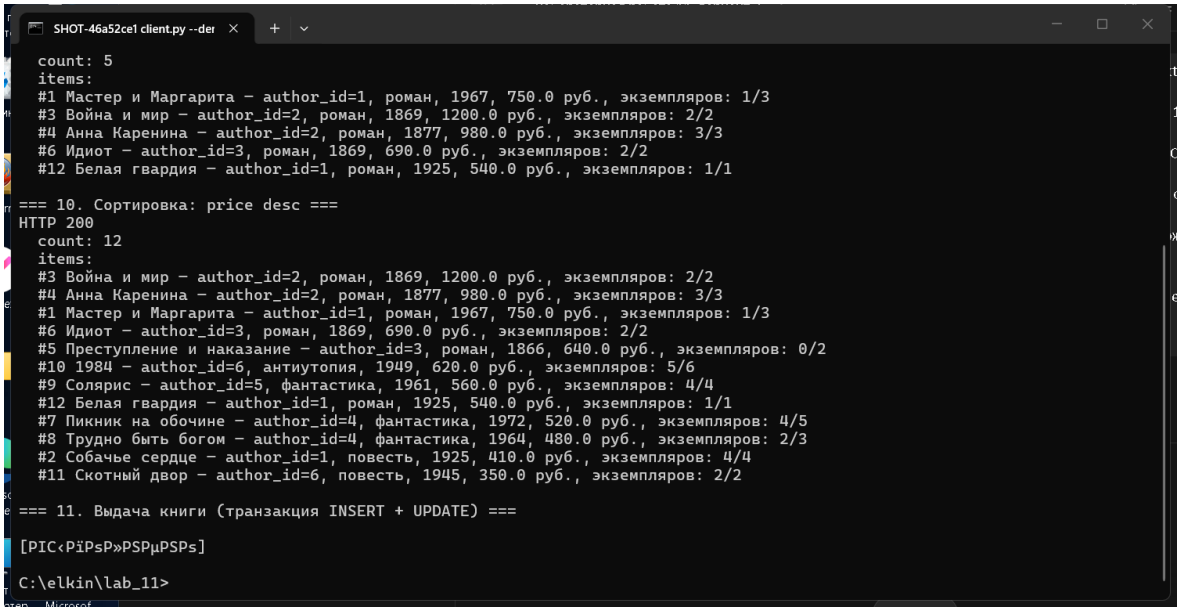



Рисунок 11.8 — Выдача книги (INSERT + UPDATE) и возврат книги (UPDATE + UPDATE)

## NoSQL: Redis

В качестве NoSQL-хранилища выбран Redis. Выбор обоснован характером данных: история поиска и кэш отдельных книг — это небольшие значения по ключу с временем жизни, для которых не нужны транзакции и

JOIN. Redis хранит такие данные в памяти и выполняет GET/SET/INCR за доли миллисекунды, тогда как реляционная БД для этой задачи избыточна.

На рабочей машине сервер Redis не установлен (официальной сборки для Windows нет), поэтому развёрнут локальный Redis-совместимый сервер mini_redis.py, реализующий протокол RESP и команды PING, SET (с EX), GET, DEL, EXPIRE, TTL, INCR, KEYS, DBSIZE, FLUSHDB. Приложение работает через штатный клиент redis-py, поэтому переход на промышленный Redis сводится к изменению host и port в config.py.

Структура ключей и время жизни приведены в таблице 11.5.

*Таблица 11.5 — Структура ключей Redis и время жизни записей*

| Ключ | Тип данных | TTL | Назначение |
|---|---|---|---|
| library:book:{id} | строка (JSON книги) | 60 с | кэш чтения книги по идентификатору |
| library:search:count:{q} | счётчик (INCR) | 3600 с | история и частота поисковых запросов |
| library:demo:* | строка/счётчик | 2–90 с | демонстрация команд SET/GET/DEL/EXPIRE/INCR |

Реализованы пять операций Redis: SET (запись значения с TTL), GET (чтение), DEL (удаление ключа), EXPIRE (установка времени жизни) и INCR (инкремент счётчика поиска). Чтение книги реализовано по паттерну cache-aside:

*Листинг 11.7 — Cache-aside при чтении книги (service.py)*

```
def get_book(self, book_id: int) -> Book:
    key = f"book:{book_id}"
    cached = self.cache.get_product_json(key)      # 1. проверяем Redis
    if cached is not None:
        return Book(**json.loads(cached))          # попадание: ответ из кэша


    book = self.books.get_by_id(book_id)           # 2. промах: SELECT из БД
    if book is None:
        raise NotFoundError(f"Book with id={book_id} not found")


    self.cache.set(key, json.dumps(book.to_dict(), ensure_ascii=False))  # 3. кладём в кэш
    return book
```

Инвалидация выполняется при любом изменении данных: после POST, PUT, PATCH, DELETE и операций выдачи/возврата ключ book:{id} удаляется, поэтому следующий GET читает свежие данные из БД. TTL ограничивает время жизни записи: даже без явной инвалидации устаревшее значение

исчезнет через 60 секунд, что важно для изменяемых данных (цена, количество экземпляров).

Результаты эксперимента с Redis приведены на рисунке 11.9: показаны базовые команды, промах и попадание кэша (2,26 мс против 0,14 мс), инвалидация после PATCH, истечение TTL и история поиска.

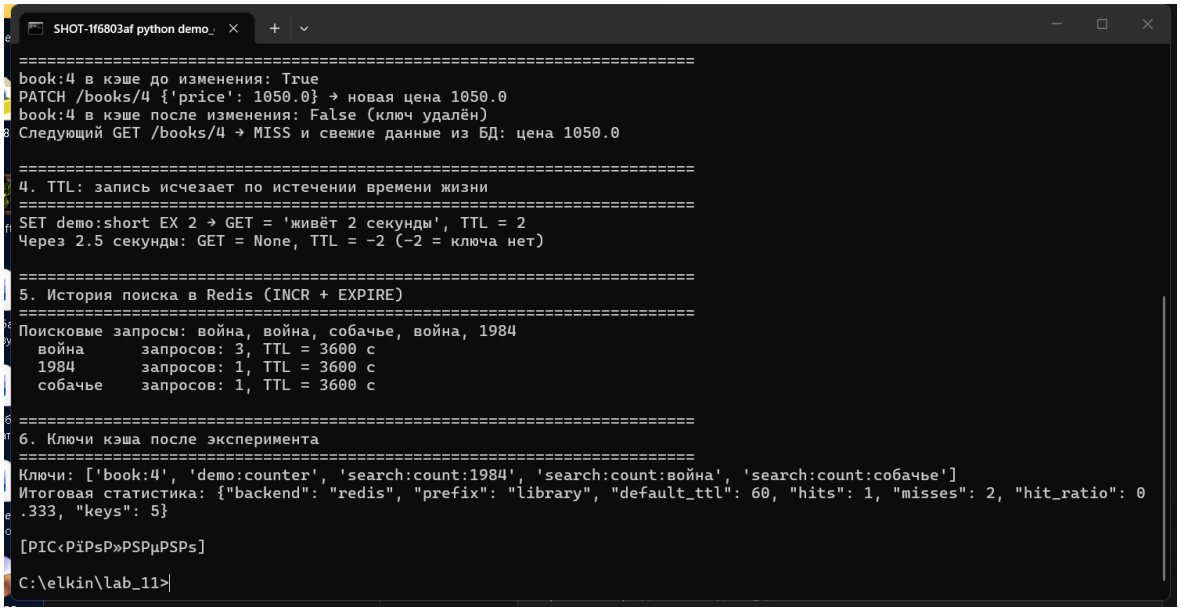



Рисунок 11.9 — Работа Redis: команды, cache-aside (MISS/HIT), инвалидация, TTL, история поиска

На рисунке 11.10 показано, что REST-запрос поиска увеличивает счётчик в Redis, а GET /books/search-history возвращает историю запросов и статистику кэша.

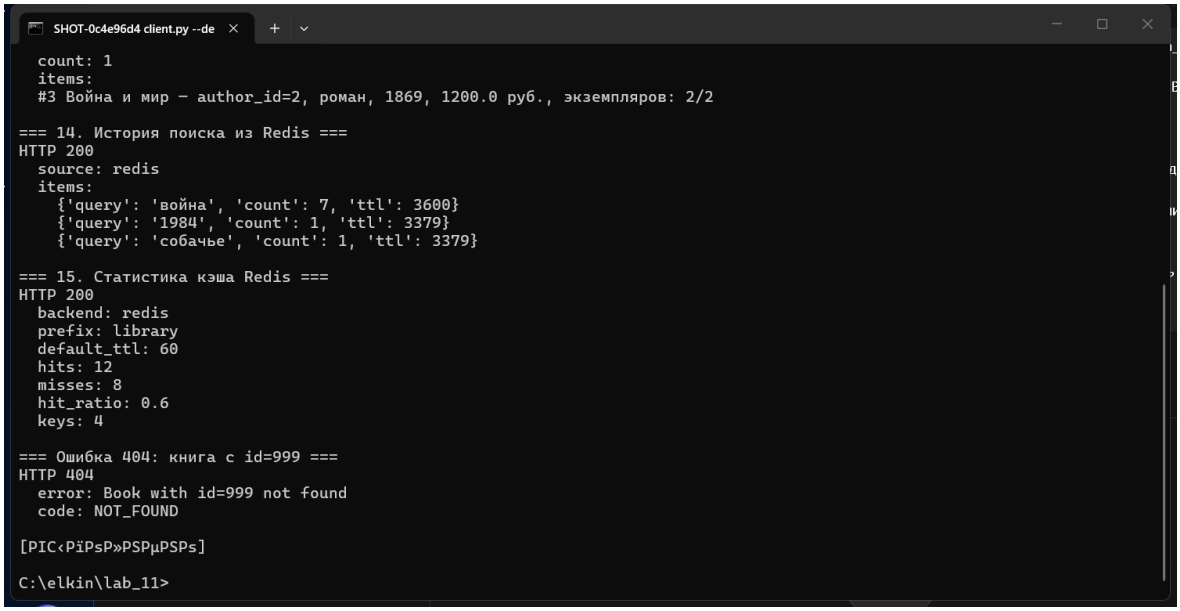



Рисунок 11.10 — Поиск книги, история поиска и статистика кэша через REST API

**CDN**

В качестве CDN-контента используются обложки книг и файлы оформления каталога. Поскольку внешний CDN в лабораторной работе недоступен, применена разрешённая методичкой локальная имитация: origin-сервер (порт 8080) отдаёт файлы из каталога static, а edge-сервер CDN (порт 8081) кэширует их и добавляет заголовки X-Cache и Age.

Размещённые ресурсы и политика кэширования приведены в таблице 11.6.

*Таблица 11.6 — Статические ресурсы, размещённые в CDN*

| Ресурс | URL origin | URL CDN | Cache-Control |
|---|---|---|---|
| изображения (12 обложек) | /images/cover-01.png … cover-12.png | http://127.0.0.1:8081/images/cover-01.png | public, max-age=86400 |
| стили | /css/style.css | http://127.0.0.1:8081/css/style.css | public, max-age=3600 |
| скрипт каталога | /js/app.js | http://127.0.0.1:8081/js/app.js | public, max-age=3600 |

Для каждого ресурса исследованы URL исходного и CDN-ресурса, HTTP-заголовки (Content-Type, Content-Length, Cache-Control, Age, X-Cache, X-Served-By), время получения и размер ответа, а также разница между первым и последующим запросами (рисунок 11.11).

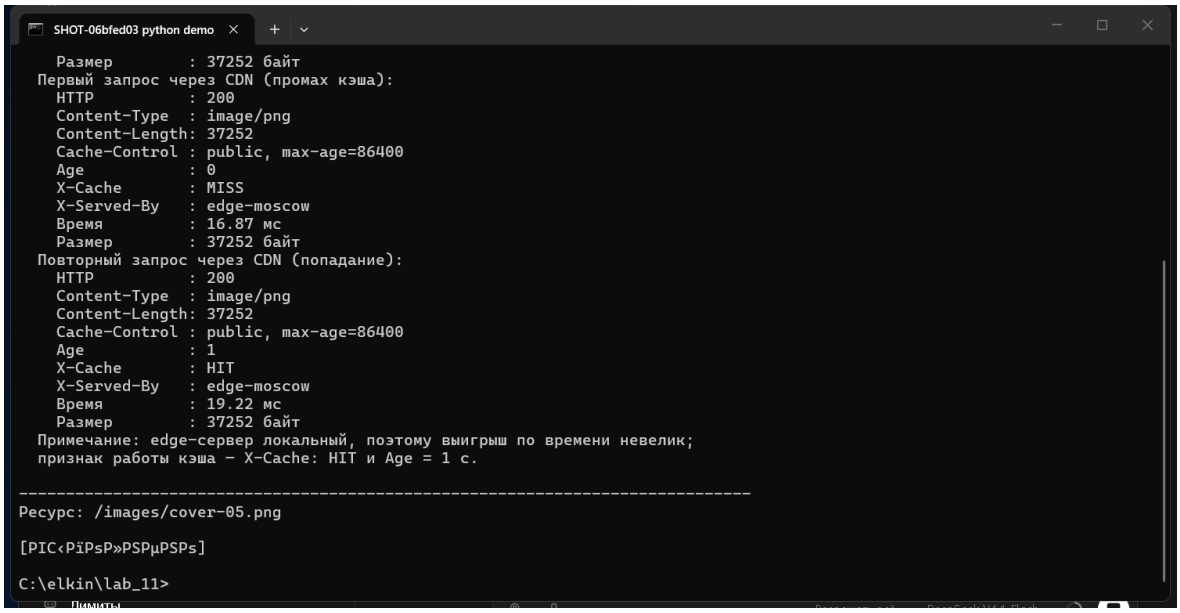



Рисунок 11.11 — Заголовки ответа при прямой загрузке с origin и при загрузке через CDN

Первый запрос к edge-серверу возвращает X-Cache: MISS и Age: 0 — ресурс берётся с origin и сохраняется на edge. Повторный запрос возвращает X-Cache: HIT и Age: 1 — ресурс отдан из кэша CDN без обращения к origin. Заголовок Cache-Control: public, max-age=86400 означает, что обложка может кэшироваться 24 часа, а стили и скрипты — 1 час (max-age=3600). Состояние кэша edge-сервера показано на рисунке 11.12.

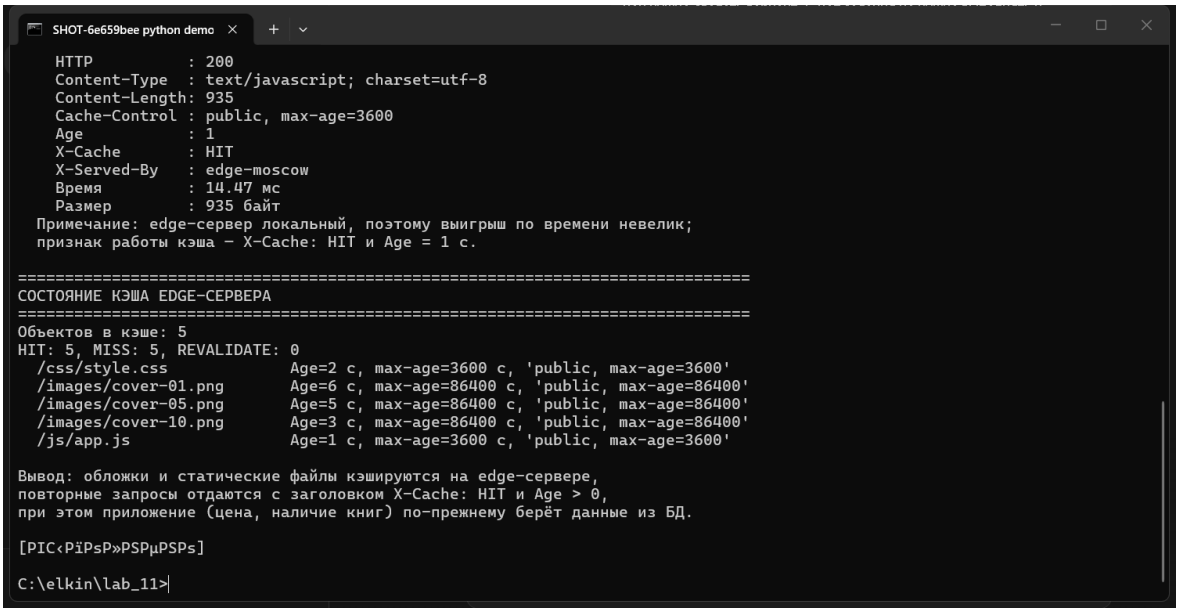



Рисунок 11.12 — Состояние кэша edge-сервера: объекты, Age, max-age, количество HIT и MISS

Принципиальное различие трёх уровней хранения в выполненной работе: цена и количество экземпляров книги — изменяемые данные, они хранятся в БД и кэшируются не более 60 секунд; история поиска — временные данные, она живёт в Redis; обложка книги — статический файл, он отдаётся через CDN и кэшируется на сутки. CDN не заменяет БД: он не умеет фильтровать, объединять и изменять данные.

**Веб-страница каталога**

REST API возвращает данные в формате JSON, поэтому для наглядной демонстрации работы приложения добавлена витрина каталога: маршрут GET / отдаёт шаблон templates/index.html, а новый endpoint GET /catalog возвращает книги вместе с ФИО автора и ссылкой на обложку. Стили и скрипт страницы подключаются с edge-сервера CDN, поэтому открытая в браузере страница

одновременно показывает работу всех трёх уровней хранения: данные о книгах приходят из SQLite, обложки — из CDN, а чтение книги по идентификатору проходит через кэш Redis.

На странице реализованы поиск по названию (параметр q запроса GET /catalog), фильтр «только доступные» (available=true) и сортировка по названию, цене, году или автору. Карточки книг показывают обложку, название, автора, жанр, год, цену и количество доступных экземпляров. При выборе карточки открывается панель с данными запроса GET /books/{id}/details: автор, теги из связи N:M и наличие. Кнопка «Оформить выдачу» вызывает POST /loans, то есть ту же транзакцию, что и консольный клиент: количество экземпляров на карточке сразу уменьшается, потому что после COMMIT кэш книги инвалидируется.

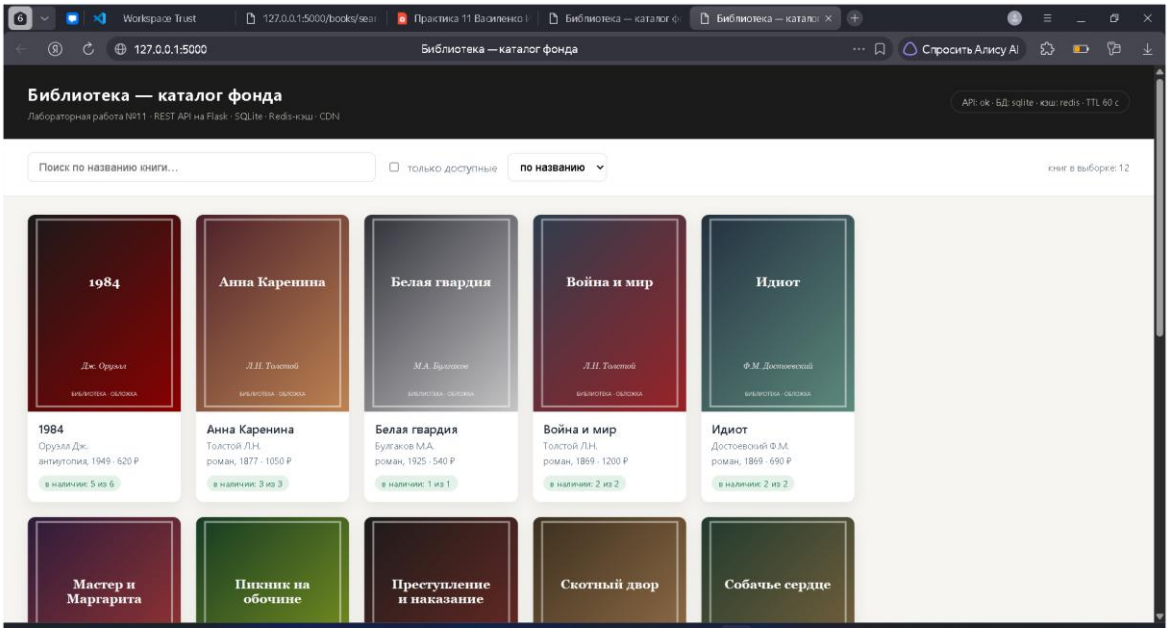



Рисунок 11.13 — Страница каталога: данные из SQLite, обложки из CDN, выдача книги через POST /loans

Страница не заменяет REST API и не содержит собственной логики работы с данными: она обращается к тем же endpoints, что и консольный клиент, а вся бизнес-логика остаётся в слое Service, поэтому поведение приложения одинаково для браузера и для HTTP-клиента.

## Сравнение технологий

Сравнительная характеристика использованных систем хранения приведена в таблице 11.7.

*Таблица 11.7 — Сравнительная характеристика систем хранения*

| Характеристика | PostgreSQL / SQLite | MongoDB | Redis | CDN |
|---|---|---|---|---|
| Основное назначение | основное хранилище данных приложения | документное хранилище | кэш, счётчики, сессии | доставка статического контента |
| Структура | таблицы, строки, столбцы | коллекции документов (BSON) | ключ → значение | файлы по URL |
| Хранение | на диске, с журналом транзакций | на диске | в оперативной памяти | на edge-серверах |
| Запросы | SQL: JOIN, GROUP BY, подзапросы | find/aggregate | GET/SET/INCR по ключу | HTTP GET файла |
| Транзакции | полноценные, ACID | много-документные транзакции | атомарные команды, без ACID-набора | нет |
| Кэширование | нет встроенного кэша чтения | кэш в памяти (WiredTiger) | сам является кэшем, TTL | кэш с Cache-Control и Age |
| Масштабирование | репликация, шардирование | шардирование, репликация | кластер, реплики | сеть edge-серверов |
| Типичные данные | заказы, книги, читатели, платежи | профили, каталоги с гибкой схемой | сессии, счётчики, история поиска | изображения, CSS, JS, видео |
| Когда использовать | целостность и связи важнее скорости записи | структура документов меняется | нужны очень быстрые чтения по ключу | много статики и географически распределённые пользователи |

## Тестирование

Написано 51 автоматический тест на pytest. Тесты не требуют запущенных серверов: используется временная БД, создаваемая из schema.sql и seed.sql, и тестовый кэш с тем же интерфейсом, что у RedisCache. Состав тестов приведён в таблице 11.8.

*Таблица 11.8 — Состав автоматических тестов*

| Файл | Тестов | Что проверяется |
|---|---|---|
| tests/test_repository.py | 15 | SQL-запросы, фильтры, сортировка, JOIN, связь N:M, GROUP BY/HAVING, ограничения внешних ключей, попытка SQL-инъекции |
| tests/test_service.py | 13 | бизнес-логика, cache-aside, инвалидация, валидация полей, поиск и история поиска, статистика |
| tests/test_transaction.py | 9 | выдача и возврат книги, откат при отсутствии экземпляров, повторный возврат, инвалидация кэша |
| tests/test_api.py | 14 | все endpoints REST API, коды 201/204/400/404/409, чтение и запись через HTTP |

Результат прогона тестов приведён на рисунке 11.14: все 51 тест пройдены.

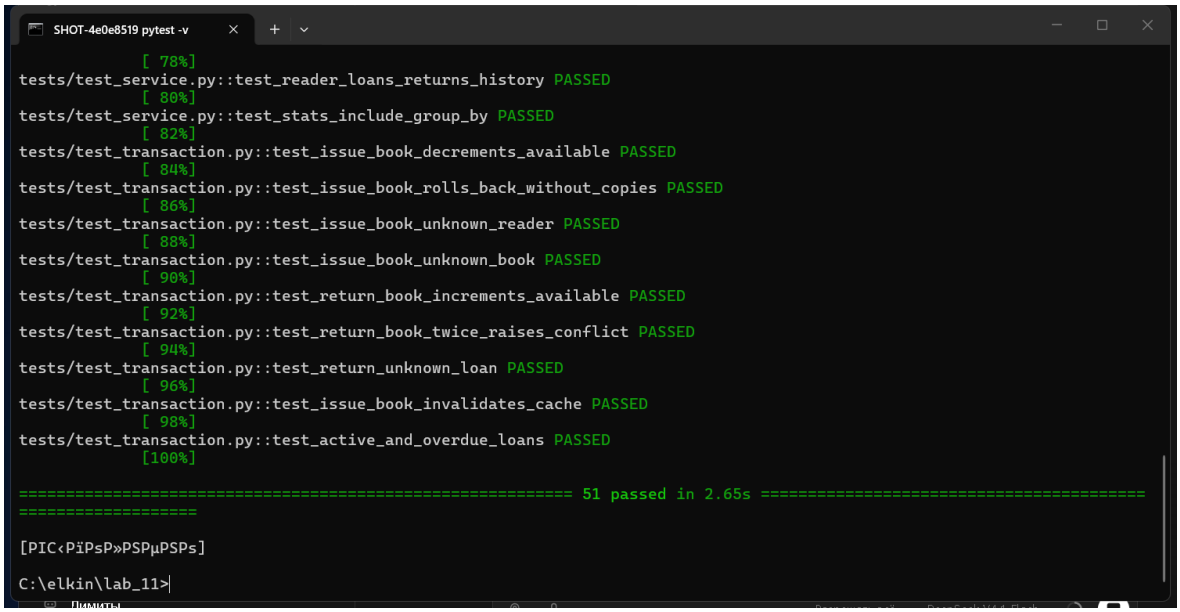



Рисунок 11.14 — Результат выполнения тестов pytest

**Вывод**

В ходе лабораторной работы разработана система хранения данных для предметной области «Библиотека» (вариант 2). Спроектирована реляционная схема из шести таблиц с первичными и внешними ключами, связями 1:N и N:M через промежуточную таблицу book_tags; схема приведена к 3НФ. Подготовлены schema.sql, seed.sql и 15 SQL-запросов, охватывающих SELECT, WHERE, ORDER BY, JOIN, LEFT JOIN, GROUP BY, HAVING, агрегатные функции, подзапросы, INSERT, UPDATE и DELETE. Создано пять индексов, назначение которых подтверждено планами выполнения запросов.

Приложение из лабораторной работы №10 переведено на работу с БД: все endpoints обращаются к данным через слой Repository, SQL-запросы параметризованы, что исключает SQL-инъекции. Реализована транзакционная бизнес-операция выдачи книги: добавление записи о выдаче и уменьшение количества свободных экземпляров выполняются как единое целое, а при ошибке выполняется ROLLBACK, и состояние БД не меняется.

Дополнительно реализованы Redis-кэш по паттерну cache-aside с TTL 60 секунд, автоматической инвалидацией при изменении данных и историей

поиска на счётчиках INCR, а также раздача статических ресурсов через CDN с заголовками Cache-Control, Age и X-Cache. Проведённые эксперименты показали разницу между уровнями хранения: изменяемые данные (цена, наличие) должны поступать из БД, временные (история поиска) — из Redis, статические (обложки) — из CDN. Написано 51 автоматический тест, все тесты проходят.

Главный результат работы — понимание того, что современное приложение использует несколько систем хранения одновременно, и выбор конкретной системы определяется характером данных, требованиями к целостности и скорости доступа.